

PL/SQL CURSORS

Code Problem Workbook

Learn the patterns used to solve cursor programs

This short edition covers:

1. Cursor meaning, context area, and active set
2. Implicit and explicit cursors
3. Declare, open, fetch, and close
4. Cursor attributes and %ROWTYPE
5. Cursor FOR loops and cursor parameters
6. Nested cursors, FOR UPDATE, and WHERE CURRENT OF
7. Debugging patterns and solved programs

FOCUS: Memorize the manual cursor pattern and cursor attributes first. Most cursor questions are variations of those two ideas.

What You Need to Study

Priority	Topic	What to remember
1	Explicit cursor lifecycle	DECLARE, OPEN, FETCH, EXIT, PROCESS, CLOSE
2	Attributes	%FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN
3	Cursor FOR loop	OPEN, FETCH, and CLOSE happen automatically
4	Parameterized cursor	Pass filter values when the cursor opens
5	%ROWTYPE	One record matches the cursor select list
6	FOR UPDATE	Lock rows and use WHERE CURRENT OF

Minimal practice data

All examples use this small EMPLOYEES table.

EMP_ID	EMP_NAME	JOB	SALARY	DEPT_ID
101	Asha	MANAGER	9000	20
102	Ravi	DEVELOPER	6000	20
103	Meena	DEVELOPER	6500	20
104	Kiran	SALESMAN	4000	10
105	Sana	MANAGER	8000	10
106	Priya	CLERK	3000	30

SETUP SCRIPT

```
CREATE TABLE employees (  
  emp_id NUMBER PRIMARY KEY,  
  emp_name VARCHAR2(30),  
  job VARCHAR2(20),  
  salary NUMBER,  
  dept_id NUMBER  
);  
  
INSERT INTO employees VALUES (101, 'Asha', 'MANAGER', 9000, 20);  
INSERT INTO employees VALUES (102, 'Ravi', 'DEVELOPER', 6000, 20);  
INSERT INTO employees VALUES (103, 'Meena', 'DEVELOPER', 6500, 20);  
INSERT INTO employees VALUES (104, 'Kiran', 'SALESMAN', 4000, 10);  
INSERT INTO employees VALUES (105, 'Sana', 'MANAGER', 8000, 10);  
INSERT INTO employees VALUES (106, 'Priya', 'CLERK', 3000, 30);  
COMMIT;  
SET SERVEROUTPUT ON;
```

Cursor Basics

Oracle creates a context area while processing a SQL statement. The rows returned by a query form the active set. A cursor is the handle or pointer used to work with that area and process its rows.

ANALOGY: The active set is a stack of employee cards. The cursor is your finger moving through the cards one at a time.

Types of cursors

Feature	Implicit cursor	Explicit cursor
Created by	Oracle automatically	The programmer
Used for	DML and SELECT INTO	Multi-row queries
Name	SQL	A declared name such as c_emp
Control	Oracle manages it	Program controls it

Only the adjacent PL/SQL you need

MINIMUM PL/SQL PATTERN

```
DECLARE
  v_total NUMBER := 0;           -- variable
BEGIN
  FOR i IN 1..3 LOOP             -- loop
    IF i > 1 THEN                 -- condition
      v_total := v_total + i;     -- assignment
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_total); -- output
EXCEPTION
  WHEN OTHERS THEN               -- error handling
    DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

For cursor problems, understand variables, assignment with `:=`, `LOOP` and `EXIT`, `IF`, `DBMS_OUTPUT`, `%TYPE`, `%ROWTYPE`, and the `EXCEPTION` section. Other PL/SQL topics can wait.

Implicit Cursors

Oracle automatically creates an implicit cursor for INSERT, UPDATE, DELETE, MERGE, and single-row SELECT INTO statements. Refer to the latest implicit cursor with SQL.

Attribute	Meaning
SQL%ROWCOUNT	Number of rows affected
SQL%FOUND	TRUE if at least one row was affected
SQL%NOTFOUND	TRUE if no row was affected
SQL%ISOPEN	Always FALSE because Oracle closes it automatically

MOST IMPORTANT IMPLICIT CURSOR EXAMPLE

```
BEGIN
  UPDATE employees
  SET salary = salary + 500
  WHERE dept_id = 20;

  DBMS_OUTPUT.PUT_LINE(
    SQL%ROWCOUNT || ' row(s) updated'
  );
END;
/
```

WATCH OUT: Check SQL cursor attributes immediately after the SQL statement. Another SQL statement changes which operation they describe.

Explicit Cursor Pattern

Step	Purpose	Syntax
Declare	Name the cursor and query	CURSOR c IS SELECT ...;
Open	Execute query and create active set	OPEN c;
Fetch	Copy one row into variables or record	FETCH c INTO ...;
Close	Release cursor resources	CLOSE c;

MEMORIZE THIS PROGRAM

```

DECLARE
  CURSOR c_emp IS
    SELECT emp_id, emp_name, salary
    FROM employees
    ORDER BY emp_id;

  v_id      employees.emp_id%TYPE;
  v_name    employees.emp_name%TYPE;
  v_salary  employees.salary%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_id, v_name, v_salary;
    EXIT WHEN c_emp%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      v_id || ' ' || v_name || ' ' || v_salary
    );
  END LOOP;
  CLOSE c_emp;
END;
/

```

MEMORY RULE: D-O-F-C: Declare, Open, Fetch, Close. Inside the loop: Fetch, Exit, Process.

WATCH OUT: Test %NOTFOUND immediately after FETCH. A failed final fetch leaves the old values in the target variables.

Explicit Cursor Attributes

Attribute	After OPEN, before FETCH	Successful FETCH	Failed FETCH
c%ISOPEN	TRUE	TRUE	TRUE until CLOSE
c%FOUND	NULL	TRUE	FALSE
c%NOTFOUND	NULL	FALSE	TRUE
c%ROWCOUNT	0	Rows fetched so far	Unchanged

Before OPEN, referencing %FOUND, %NOTFOUND, or %ROWCOUNT raises INVALID_CURSOR. %ISOPEN is FALSE before OPEN and after CLOSE.

Use a cursor record

EXAMPLE

```

DECLARE
  CURSOR c_emp IS
    SELECT emp_id, emp_name, salary
    FROM employees
    WHERE dept_id = 20;

  r_emp c_emp%ROWTYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO r_emp;
    EXIT WHEN c_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r_emp.emp_name);
  END LOOP;
  CLOSE c_emp;
END;
/

```

KEY POINT: %TYPE declares one matching value. %ROWTYPE declares one record containing all table columns or all columns selected by a cursor.

Cursor FOR Loop

A cursor FOR loop automatically opens the cursor, fetches every row into a loop record, and closes the cursor.

NAMED CURSOR FOR LOOP

```
DECLARE
    CURSOR c_emp IS
        SELECT emp_name, salary
        FROM employees
        WHERE dept_id = 20;
BEGIN
    FOR r_emp IN c_emp LOOP
        DBMS_OUTPUT.PUT_LINE(
            r_emp.emp_name || ' ' || r_emp.salary
        );
    END LOOP;
END;
/
```

INLINE CURSOR FOR LOOP

```
BEGIN
    FOR r_emp IN (
        SELECT emp_name, salary
        FROM employees
        ORDER BY salary DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(r_emp.emp_name);
    END LOOP;
END;
/
```

EXAM LINE: The cursor FOR loop handles OPEN, FETCH, and CLOSE implicitly. The loop record exists only inside the loop.

Parameterized and Nested Cursors

Parameterized cursor

EXAMPLE

```
DECLARE
  CURSOR c_emp (p_dept employees.dept_id%TYPE) IS
    SELECT emp_name, salary
    FROM employees
    WHERE dept_id = p_dept;
BEGIN
  FOR r_emp IN c_emp(20) LOOP
    DBMS_OUTPUT.PUT_LINE(r_emp.emp_name);
  END LOOP;
END;
/
```

- Cursor parameters use IN mode.
- The argument is evaluated when the cursor opens.
- Parameters make one cursor reusable for different result sets.

Nested cursor pattern

EXAMPLE

```
DECLARE
  CURSOR c_dept IS
    SELECT DISTINCT dept_id FROM employees;

  CURSOR c_emp (p_dept NUMBER) IS
    SELECT emp_name FROM employees
    WHERE dept_id = p_dept;
BEGIN
  FOR r_dept IN c_dept LOOP
    DBMS_OUTPUT.PUT_LINE('Department ' || r_dept.dept_id);
    FOR r_emp IN c_emp(r_dept.dept_id) LOOP
      DBMS_OUTPUT.PUT_LINE('  ' || r_emp.emp_name);
    END LOOP;
  END LOOP;
END;
/
```

The outer cursor selects departments. For every department, the inner cursor selects its employees.

FOR UPDATE and WHERE CURRENT OF

FOR UPDATE locks the selected rows. WHERE CURRENT OF updates or deletes the row most recently fetched by that cursor.

EXAMPLE

```
DECLARE
  CURSOR c_emp IS
    SELECT emp_id, salary
    FROM employees
    WHERE salary < 5000
    FOR UPDATE OF salary;
BEGIN
  FOR r_emp IN c_emp LOOP
    UPDATE employees
      SET salary = salary + 200
      WHERE CURRENT OF c_emp;
  END LOOP;
  COMMIT;
END;
```

- FOR UPDATE locks qualifying rows when the cursor opens.
- WHERE CURRENT OF avoids repeating the cursor's search condition.
- COMMIT or ROLLBACK releases the locks.
- Do not COMMIT while continuing to fetch from a FOR UPDATE cursor.

PREFERRED APPROACH: If one UPDATE statement can perform the complete change, use that statement instead of a cursor. Use a cursor only when each row needs separate procedural work.

Common Errors and Cheat Sheet

Problem	Cause	Correction
Last row printed twice	Process occurs before %NOTFOUND test	Fetch, exit, then process
CURSOR_ALREADY_OPEN	OPEN called twice	Close before reopening
INVALID_CURSOR	Fetch or close before OPEN	Follow the lifecycle
Fetch mismatch	SELECT list and targets differ	Use matching variables or %ROWTYPE
No output	SERVEROUTPUT off or no matching rows	Enable output and test the query

The two patterns to memorize

QUICK REVISION

```
-- Manual cursor
OPEN c;
LOOP
    FETCH c INTO r;
    EXIT WHEN c%NOTFOUND;
    -- process r
END LOOP;
CLOSE c;

-- Cursor FOR loop
FOR r IN c LOOP
    -- process r
END LOOP;
```

Question	Answer
Did FETCH return a row?	c%FOUND
Did FETCH fail?	c%NOTFOUND
How many rows fetched?	c%ROWCOUNT
Is the cursor open?	c%ISOPEN

Solved Cursor Problems

Problem 1: count selected rows

SOLUTION

```
DECLARE
  CURSOR c_emp IS
    SELECT emp_id FROM employees WHERE dept_id = 20;
  v_id employees.emp_id%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_id;
    EXIT WHEN c_emp%NOTFOUND;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Total: ' || c_emp%ROWCOUNT);
  CLOSE c_emp;
END;
/
```

Problem 2: calculate a total with a cursor

SOLUTION

```
DECLARE
  CURSOR c_emp IS SELECT salary FROM employees;
  v_salary employees.salary%TYPE;
  v_total NUMBER := 0;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_salary;
    EXIT WHEN c_emp%NOTFOUND;
    v_total := v_total + v_salary;
  END LOOP;
  CLOSE c_emp;
  DBMS_OUTPUT.PUT_LINE('Total salary: ' || v_total);
END;
/
```

Problem 3: classify every fetched row

SOLUTION

```
BEGIN
  FOR r_emp IN (
    SELECT emp_name, salary FROM employees ORDER BY salary DESC
  ) LOOP
    IF r_emp.salary >= 7000 THEN
      DBMS_OUTPUT.PUT_LINE(r_emp.emp_name || ': HIGH');
    ELSE
      DBMS_OUTPUT.PUT_LINE(r_emp.emp_name || ': NORMAL');
    END IF;
  END LOOP;
END;
/
```

Problem 4: process only the first three rows

SOLUTION

```
DECLARE
  CURSOR c_emp IS
    SELECT emp_name, salary FROM employees ORDER BY salary DESC;
BEGIN
  FOR r_emp IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(r_emp.emp_name || ' ' || r_emp.salary);
    EXIT WHEN c_emp%ROWCOUNT = 3;
  END LOOP;
END;
/
```

Problem 5: close a manual cursor after an error

SOLUTION

```
DECLARE
  CURSOR c_emp IS SELECT emp_id FROM employees;
  v_id employees.emp_id%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_id;
    EXIT WHEN c_emp%NOTFOUND;
    -- row processing
  END LOOP;
  CLOSE c_emp;
EXCEPTION
  WHEN OTHERS THEN
    IF c_emp%ISOPEN THEN CLOSE c_emp; END IF;
    RAISE;
END;
/
```

More Programming Problems

Q1. Display all employees using an explicit cursor

ANSWER

```
DECLARE
  CURSOR c_emp IS SELECT emp_name, salary FROM employees;
  r_emp c_emp%ROWTYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO r_emp;
    EXIT WHEN c_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r_emp.emp_name || ' ' || r_emp.salary);
  END LOOP;
  CLOSE c_emp;
END;
/
```

Q2. Display employees of a supplied department

ANSWER

```
DECLARE
  CURSOR c_emp (p_dept NUMBER) IS
    SELECT emp_name FROM employees WHERE dept_id = p_dept;
BEGIN
  FOR r_emp IN c_emp(20) LOOP
    DBMS_OUTPUT.PUT_LINE(r_emp.emp_name);
  END LOOP;
END;
/
```

Final Revision Page

Remember these lines

- Cursor: pointer to Oracle's context area.
- Active set: rows returned by the cursor query.
- Implicit cursor name: SQL.
- Explicit lifecycle: declare, open, fetch, close.
- Safe loop order: fetch, exit when %NOTFOUND, process.
- %ROWCOUNT: rows affected or fetched.
- Cursor FOR loop: automatic open, fetch, and close.
- %ROWTYPE: record matching a table or cursor row.
- Parameterized cursor: reusable cursor with IN values.
- FOR UPDATE locks rows. WHERE CURRENT OF changes the fetched row.

One complete answer pattern

WRITE THIS FROM MEMORY

```
DECLARE
  CURSOR c IS SELECT column_list FROM table_name;
  r c%ROWTYPE;
BEGIN
  OPEN c;
  LOOP
    FETCH c INTO r;
    EXIT WHEN c%NOTFOUND;
    -- process the current row
  END LOOP;
  CLOSE c;
END;
/
```

FINAL TIP: Practise the explicit cursor program three times without looking. Then practise one cursor FOR loop and one parameterized cursor.